

GESTÃO DE CLUSTERS,
SEGURANÇA QUÂNTICA E
COLABORAÇÃO MULTIAGENTE

CRIPTOGRAFIA QUÂNTICA: SEGURANÇA E A NOVA FRONTEIRA DOS DADOS



4

LISTA DE FIGURAS

Figura 1 - Circuito de Simon.....	20
Figura 2 - Primeira parte do Circuito QPE.....	28
Figura 3 - Circuito do Algoritmo de Order-Finding.....	30
Figura 4 - Circuito do Algoritmo de Shor	38
Figura 5 - Resultados do Algoritmo de Shor.....	39
Figura 6 - Resultados do Protocolo BB84	53



LISTA DE QUADROS

Quadro 1 - Comparação entre PQC e QKD	58
---	----

EMANIP

LISTA DE TABELAS

Tabela 1 - Exemplo de qubits enviados por Alice.....43

Tabela 2 - Exemplo de bits medidos por Bob.....44

Tabela 3 - Exemplo de Sifted Key46



LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Algoritmo de Shor: MDC	33
Código-fonte 2 - Algoritmo de Shor: Oráculo	34
Código-fonte 3 - Algoritmo de Shor: Circuito	35
Código-fonte 4 - Algoritmo de Shor: Pós-processamento	37
Código-fonte 5 - Algoritmo de Shor: Execução	38
Código-fonte 6 - Protocolo BB84: Setup	50
Código-fonte 7 - Protocolo BB84: Circuito	51
Código-fonte 8 - Protocolo BB84: Simulação	52
Código-fonte 9 - Protocolo BB84: Sifting	52

SUMÁRIO

1 CRIPTOGRAFIA QUÂNTICA: SEGURANÇA E A NOVA FRONTEIRA DOS DADOS	7
1.1 Introdução	7
1.2 A Ameaça Quântica	7
1.3 A Promessa Quântica	8
1.4 Computação Quântica na Segurança da Informação.....	9
2 A FRAGILIDADE DA CRIPTOGRAFIA CLÁSSICA.....	10
2.1 O Dilema da Troca de Chaves	11
2.2 RSA e Fatoração de Inteiros	12
2.2.1 Geração das Chaves (Bob)	12
2.2.2 Criptografia (Alice).....	14
2.2.3 Descryptografia (Bob).....	14
2.2.4 Exemplo Prático	15
2.2.5 Intercepção da Mensagem (Eve)	16
3 QUEBRANDO CÓDIGOS COM ALGORITMOS QUÂNTICOS.....	18
3.1 O Problema de Simon	18
3.1.1 Análise do Algoritmo de Simon	21
3.1.2 Camada de Pós-Processamento Clássico	23
3.2 O Algoritmo de Shor e o Fim da Era RSA	24
3.2.1 Encontrando Fatores Primos.....	25
3.2.2 Circuito Quântico para Descoberta de Período	27
3.2.3 Pós-processamento Clássico	31
3.2.4 Algoritmo de Shor com Qiskit	32
4 INTRODUÇÃO À CRIPTOGRAFIA QUÂNTICA.....	40
4.1 Novo Paradigma de Segurança	40
4.2 Protocolo BB84	41
4.2.1 Etapa 1: Envio (Alice)	42
4.2.2 Etapa 2: Medição (Bob).....	43
4.2.3 Etapa 3: Sifting	45
4.2.4 Etapa 4: Detecção de Espionagem	46
4.2.5 Etapa 5: Reconciliação e Amplificação de Privacidade	48
4.3 Protocolo BB84 com Qiskit.....	49
5 O CENÁRIO PÓS-QUÂNTICO	54
5.1 Vulnerabilidades da QKD	54
5.1.1 Photon Number Splitting.....	55
5.1.2 Decoy States	56
5.2 Criptografia Pós-Quântica	57
5.3 Comparação entre PQC e QKD	58
CONCLUSÃO	60
REFERÊNCIAS.....	62

1 CRIPTOGRAFIA QUÂNTICA: SEGURANÇA E A NOVA FRONTEIRA DOS DADOS

1.1 Introdução

Nos capítulos até então, estudamos os fundamentos da Computação Quântica, desde sua base matemática até a construção de circuitos e a implementação de algoritmos que demonstram uma nova forma de processar informações.

Vimos como qubits, em vez de nos limitar aos estados 0 ou 1, podem existir em estados de superposição e como fenômenos como o entrelaçamento criam correlações mais fortes entre múltiplos qubits do que equivalentes clássicos.

Com algoritmos como o de **Deutsch-Jozsa e Grover**, testemunhamos o potencial para resolver problemas específicos com uma eficiência muito maior do que qualquer método tradicional.

Toda a teoria vista nestes capítulos servirá para entendermos as aplicações da Computação Quântica em alguns dos campos mais críticos da tecnologia moderna. Neste capítulo, abordaremos a segurança da informação, enquanto o próximo discutirá o Machine Learning.

1.2 A Ameaça Quântica

A capacidade da Computação Quântica de processar informações de maneiras radicalmente novas não é apenas uma promessa para o futuro da ciência; ela representa uma verdadeira “faca de dois gumes” para o presente.

De um lado, o avanço dos computadores quânticos representa uma ameaça para grande parte da infraestrutura de segurança que sustenta nossa sociedade digital: sistemas de chave pública, como o **RSA**, são a espinha dorsal do comércio eletrônico, das comunicações seguras e da proteção de dados sigilosos.

Sua segurança não se baseia em um conceito matemático que provamos ser inviolável, mas na dificuldade computacional de problemas como a fatoração de grandes números inteiros.

Para computadores clássicos, mesmo os mais poderosos, fatorar os números utilizados na criptografia moderna é uma tarefa considerada intratável; no entanto, o **Algoritmo de Shor** pode resolver esse problema de forma eficiente em um computador quântico, tornando esses sistemas vulneráveis.

Para um profissional de Inteligência Artificial, entender essa ameaça é crucial, visto que modelos de IA e grandes volumes de dados são frequentemente protegidos por essa mesma criptografia. A integridade dos dados de treinamento, a confidencialidade dos modelos proprietários e a privacidade dos dados de usuários dependem de um sistema de segurança forte.

A perspectiva de uma “**quebra quântica**” exige que futuros arquitetos de sistemas de IA estejam cientes dessas vulnerabilidades e preparados para a transição a novos padrões de segurança.

1.3 A Promessa Quântica

Do outro lado da lâmina, a mesma mecânica quântica que ameaça quebrar a criptografia clássica também oferece uma nova promessa: formas de comunicação com segurança incondicional, baseadas não em complexidade matemática, mas nas próprias leis da física.

Protocolos como o **BB84**, que exploram a Distribuição de Chaves Quânticas (QKD), utilizam princípios como o **Teorema da Não-Clonagem** e o fato de que a medição perturba um sistema quântico, permitindo que duas partes compartilhem uma chave secreta com a garantia de que qualquer tentativa de espionagem será detectada.

Na indústria, essa promessa abre portas para uma nova geração de comunicação segura, especialmente em setores que demandam proteção de longo prazo para informações sensíveis, como finanças, defesa e saúde.

Para um especialista em IA, a QKD pode garantir a segurança dos canais de comunicação entre componentes de um sistema de aprendizado distribuído ou proteger a transmissão de dados para inferência em nuvem, oferecendo um nível de segurança que os métodos clássicos não podem garantir.

1.4 Computação Quântica na Segurança da Informação

Neste capítulo, faremos uma introdução para esses dois lados da Computação Quântica: começaremos entendendo a **fragilidade da criptografia clássica**, com foco no problema da fatoração que sustenta o RSA.

Em seguida, construiremos a base para a principal ameaça quântica, introduzindo o **Problema de Simon** e sua evolução: o **Algoritmo de Shor**.

Depois de compreender a ameaça, falaremos sobre a **solução**, detalhando o **protocolo BB84** de Distribuição de Chaves Quânticas como um novo paradigma de segurança.

Por fim, olharemos para o futuro, discutindo as ameaças práticas aos sistemas quânticos e a ascensão da **Criptografia Pós-Quântica (PQC)**, uma iniciativa para desenvolver algoritmos clássicos que sejam resistentes tanto a ataques clássicos quanto quânticos.

2 A FRAGILIDADE DA CRIPTOGRAFIA CLÁSSICA

Agora que estabelecemos as relações da Computação Quântica com a segurança, vamos analisar o primeiro lado: a ameaça. Para compreender por que um computador quântico pode ser tão disruptivo, precisamos primeiro entender como a segurança digital funciona em nosso mundo clássico.

Quase toda a informação que protegemos, desde transações financeiras e segredos de estado até os dados de treinamento para um modelo de IA, depende de um campo da matemática chamado **criptografia**.

Historicamente, o maior desafio na comunicação secreta sempre foi o de garantir que o **emissor** “A” (que chamaremos de Alice) e o **receptor** “B” (Bob) compartilhem uma chave secreta para criptografar e descriptografar suas mensagens, sem que um **espião** “E” (Eve) a intercepte.

A criptografia moderna resolveu em grande parte esse problema com a invenção da criptografia de **chave pública** (ou **chave assimétrica**). Diferente dos métodos de chave privada, em que a mesma chave é usada para codificar e decodificar, os sistemas de chave pública, como o famoso **RSA**, baseiam sua segurança na dificuldade computacional de certos problemas matemáticos.

A ideia é criar uma “armadilha” matemática que é fácil de montar, mas extremamente difícil de desmontar sem a informação correta. No caso do RSA, essa armadilha é a multiplicação de dois números primos muito grandes.

Multiplicá-los é uma tarefa trivial para qualquer computador. No entanto, o processo inverso (pegar o resultado, que é público, e descobrir quais foram os fatores primos originais) é uma tarefa considerada intratável, até para os computadores clássicos mais poderosos do mundo.

Toda a segurança do RSA, que protege inúmeros sistemas hoje, baseia-se nesta única suposição de dificuldade computacional e é aqui que a Computação Quântica muda as regras do jogo.

2.1 O Dilema da Troca de Chaves

Para entender por que a Computação Quântica é tão relevante para a segurança, precisamos primeiro visitar duas ideias centrais da criptografia moderna: a **criptografia de chave simétrica** e a de **chave assimétrica**.

A forma mais intuitiva de comunicação secreta é a criptografia de chave simétrica, também conhecida como criptografia de chave privada. Nela, tanto Alice quanto Bob utilizam a mesma chave secreta para criptografar e descriptografar suas mensagens. Um exemplo famoso é o “**one-time pad**”, que, se usado corretamente, é comprovadamente inquebrável.

NOTA: o one-time pad funciona combinando cada letra da mensagem com uma letra de uma chave secreta, longa e totalmente aleatória. A chave deve ter o mesmo tamanho da mensagem e ser usada uma única vez. Essa aleatoriedade e uso único garantem que, sem a chave, a mensagem criptografada seja indecifrável.

No entanto, ele possui uma falha fundamental que não está no algoritmo em si, mas em sua logística: como Alice e Bob podem trocar a chave secreta entre si (ou seja, combinar qual será a chave) de forma segura?

Se eles já possuem um canal seguro para enviar a chave, por que não usar esse mesmo canal para enviar a mensagem diretamente? Este é o **dilema da distribuição de chaves**.

Para resolvê-lo, a criptografia de chave pública foi desenvolvida. Nesse modelo, cada pessoa possui um par de chaves: uma chave pública, que pode ser compartilhada livremente com qualquer pessoa, e uma chave privada, que deve ser mantida em segredo absoluto.

Se Alice quer enviar uma mensagem para Bob, ela utiliza a chave pública de Bob para criptografá-la. O diferencial do sistema está no fato de que apenas a chave privada correspondente, que só Bob possui, pode descriptografar a mensagem; dessa forma, o problema da distribuição segura da chave é contornado.

A segurança desses sistemas não depende de manter a chave de criptografia secreta (já que ela é pública), mas sim de uma “armadilha matemática”: uma função

que é fácil de calcular em uma direção (criptografar com a chave pública), mas extremamente difícil de reverter (descriptografar sem a chave privada).

2.2 RSA e Fatoração de Inteiros

O criptosistema mais famoso e amplamente utilizado baseado no princípio de chave pública é o **RSA**, nomeado em homenagem a seus inventores Ron Rivest, Adi Shamir e Leonard Adleman.

A segurança do RSA depende inteiramente de um problema matemático que, apesar de simples de enunciar, computadores clássicos têm dificuldades em resolver: o **problema da fatoração de inteiros**.

Temos que qualquer número inteiro positivo N pode ser escrito como uma multiplicação de números primos, tal que:

$$N = p \cdot q, \text{ em que } p, q \text{ são primos}$$

Dado um número inteiro N , encontrar seus fatores primos p e q é uma tarefa extremamente difícil quando N é muito grande. Em contraste, a operação inversa, que é multiplicar p e q para obter N , é trivial.

O RSA explora essa assimetria na dificuldade para criar seu par de chaves. De forma simplificada, o protocolo executa os seguintes passos:

2.2.1 Geração das Chaves (Bob)

Bob escolhe dois números primos muito grandes, p e q , que são mantidos em segredo.

Ele calcula o produto $N = pq$. O resultado N , chamado de módulo, fará parte da chave pública.

Bob também escolhe um número inteiro e (**expoente público**) que seja coprimo ao resultado de $(p - 1)(q - 1)$. Este produto é conhecido como **Função Totiente de Euler**, $\phi(N)$, definida como o número de inteiros menores do que N que são coprimos a N .

NOTA: dois números são “coprimos” (ou **primos entre si**) quando o único número inteiro positivo que consegue dividir ambos exatamente é o número 1, ou seja, o **MDC** (Máximo Divisor Comum) entre eles é 1.

Eles não precisam ser números primos individualmente. Por exemplo, 8 e 9 são coprimos, pois nenhum inteiro além do 1 divide os dois ao mesmo tempo.

Mais formalmente, podemos dizer que o expoente público deve ser um inteiro e tal que:

$$1 < e < \phi(N) \text{ e } \text{mdc}(e, \phi(N)) = 1$$

A dupla (N, e) é a **chave pública** de Bob, que pode ser livremente distribuída e será usada para criptografar a mensagem.

Por fim, ele calcula seu **expoente privado**, d , de forma que:

$$d \cdot e \equiv 1 \pmod{\phi(N)}$$

Esta “equação” pode ser interpretada pela **aritmética modular**. De forma simples, a relação:

$$a \equiv b \pmod{n}$$

Lida como “ a é **congruente** a b módulo n ”, significa que a e b têm o mesmo resto se divididos por n . Por exemplo, $10 \equiv 14 \pmod{4}$, já que:

- A divisão inteira entre 10 e 4 resulta em 2, com **resto 2**;
- A divisão inteira entre 14 e 4 resulta em 3, com **resto 2**.

A congruência também pode sempre ser “invertida”, de forma que:

$$a \equiv b \pmod{n} \leftrightarrow b \equiv a \pmod{n}$$

Isso nos facilita utilizar outra definição, que diz que $n|(a - b)$: n deve ser **divisor** de $a - b$, ou seja, $a - b$ precisa ser um **múltiplo** de n .

No exemplo anterior, vemos que a relação de congruência é verdadeira pois:

$$10 \equiv 14 \pmod{4} \leftrightarrow 14 \equiv 10 \pmod{4}$$

E $14 - 10 = 4$, que obviamente é múltiplo de 4.

Dessa forma, podemos dizer que nosso expoente privado d deve ser um certo número inteiro tal que $(de) - 1$ deve ser múltiplo de $\phi(N)$.

O valor d também será a chave privada de Bob, que deve ser mantida em segredo a todo custo, pois será utilizada para descriptografar mensagens recebidas.

2.2.2 Criptografia (Alice)

Alice, querendo enviar uma mensagem M para Bob (em que M é um número menor que N), deve utilizar a chave pública de Bob para cifrar sua mensagem, obtendo c , tal que:

$$c = M^e \bmod N$$

NOTA: neste caso, não estamos calculando a congruência, mas a operação $a \bmod b$ de fato, que equivale ao **resto de divisão** entre a e b .

Note, ainda, que os parênteses são opcionais: $a \bmod b = a \pmod{b}$

Alice pode enviar a mensagem c por qualquer canal público e inseguro, sem se preocupar com a possibilidade de a mensagem ser interceptada pela espiã Eve.

2.2.3 Descriptografia (Bob)

Bob recebe c e pode descriptografar a mensagem utilizando sua chave privada e algumas propriedades.

O **Teorema de Euler** diz que, se certos inteiros m e n forem coprimos, é válida a relação:

$$m^{\phi(n)} \pmod{n} = 1$$

Elevando ambos os lados por uma constante k , temos:

$$m^{k \cdot \phi(n)} \pmod{n} = 1^k = 1$$

Multiplicando por m ,

$$m^{k \cdot \phi(n)} \cdot m \pmod{n} = 1 \cdot m \leftrightarrow m^{k \cdot \phi(n) + 1} \pmod{n} = m$$

Uma forma de interpretar o expoente $x = k \cdot \phi(n) + 1$ é dizer que $x - 1$ é múltiplo de $\phi(n)$.

Pelas definições que utilizamos anteriormente, sabemos que a mensagem criptografada c e o módulo da chave pública N são coprimos; além disso, a multiplicação entre os expoentes público e privado, menos um, é um múltiplo de $\phi(N)$.

Juntando as relações, podemos dizer que a expressão $c^d \bmod N$ é equivalente a:

$$c^d \bmod N = M^{ed} \bmod N = M^{k\phi(N)+1} \bmod N = M$$

Ou seja, podemos utilizar $c^d \bmod N$ para descriptografar a mensagem enviada por Alice.

2.2.4 Exemplo Prático

Para mostrar o funcionamento do protocolo na prática, vamos fazer um exemplo utilizando números pequenos.

Suponha que Alice quer enviar o número “7” para Bob.

Primeiro, ele deve criar sua chave pública. Para tal, escolheu os números primos $p = 5$ e $q = 11$, de forma que:

$$N = pq = 55$$

$$\phi(N) = 4 \cdot 10 = 40$$

Como e deve ser um número entre 1 e 40, Bob escolheu $e = 3$, já que:

$$\text{mdc}(3, 40) = 1$$

Assim, ele informa à Alice que sua chave é:

$$\text{Pub} = (55, 3)$$

Então, ela criptografa sua mensagem e envia c , que será:

$$M = 7$$

$$c = 7^3 \bmod 55 = 13$$

Bob precisa então definir sua chave privada, que deve ser algum d , tal que:

$$3d \equiv 1 \pmod{40}$$

Das possibilidades, ele percebeu que $3 \cdot 27 = 81$. Como $80 = 81 - 1$ é múltiplo de 40, decidiu escolher 27 como sua chave privada:

$$\textit{Priv} = 27$$

Para decodificar a mensagem, basta calcular $c^d \bmod N$, que neste caso será:

$$13^{27} (\bmod 55) = 7$$

Assim, Bob foi capaz de recuperar a mensagem original utilizando apenas seu par de chaves.

2.2.5 Intercepção da Mensagem (Eve)

Se a espiã Eve conseguir interceptar a mensagem c antes que ela chegue a Bob, será incapaz de descryptografá-la corretamente, pois não conhece o valor de d . Para descobri-lo, Eve precisaria calcular $\phi(N)$, o que é impossível sem p e q , sendo necessário fatorar N primeiro.

Para números de N pequenos, é possível fazer até manualmente. Porém, o problema da fatoração de inteiros é computacionalmente difícil. Os algoritmos clássicos mais eficientes que resolvem esse problema, como o chamado **General Number Field Sieve** (GNFS), possuem complexidade de tempo sub exponencial. No caso do GNFS, sua complexidade é aproximadamente:

$$O\left(e^{(\ln N)^{1/3} (\ln \ln N)^{2/3}}\right)$$

Em que N é o número a ser fatorado e e é o número de Euler.

Se N for um valor grande o bastante, a execução desse algoritmo pode levar séculos em computadores clássicos, o que explica a utilidade e segurança do protocolo RSA até então.

Um algoritmo que pudesse fatorar N de forma eficiente permitiria que qualquer pessoa com acesso à chave pública pudesse calcular a chave privada, quebrando assim toda a segurança do sistema.

E é exatamente aqui que a Computação Quântica representa uma ameaça: o Algoritmo de Shor consegue resolver o problema da fatoração em **tempo polinomial**, com complexidade aproximada de:

$$O((\ln N)^3)$$

Permitindo Eve quebrar facilmente o sistema de criptografia, mesmo sem a chave privada de Bob.

EMSE

3 QUEBRANDO CÓDIGOS COM ALGORITMOS QUÂNTICOS

Ressaltando novamente, a segurança do protocolo RSA depende da suposição de que fatorar números grandes é uma tarefa difícil para computadores. A seguir, vamos introduzir os algoritmos quânticos que invalidam essa suposição, demonstrando uma vantagem computacional exponencial.

O primeiro passo será o **Problema de Simon**, uma generalização do Problema de **Deutsch-Jozsa** que, embora de menor interesse prático direto, é de imensa importância histórica e didática, pois facilita o entendimento de algoritmos mais complexos, como o **Algoritmo de Shor**, que visamos compreender antes do final do capítulo.

O Problema de Simon estabelece o paradigma de usar a **Transformada Quântica de Fourier** para extrair o “período escondido” de uma função, uma técnica que está no cerne do Algoritmo de Shor.

3.1 O Problema de Simon

Assim como os algoritmos estudados anteriormente, o Problema de Simon nos apresenta um cenário em que um algoritmo quântico demonstra uma vantagem exponencial sobre qualquer algoritmo clássico (incluindo os probabilísticos) para encontrar um tipo específico de período oculto.

Para o problema, que é novamente um problema de query com promessa, temos um oráculo $f: \Sigma^n \rightarrow \Sigma^m$, novamente utilizando o conjunto $\Sigma = \{0,1\}$.

Além disso, nos é fornecido a promessa: existe apenas uma string secreta e não nula $s \in \Sigma^n$ tal que, para qualquer par de inputs x e $y \in \Sigma^n$:

$$f(x) = f(y) \leftrightarrow (x \oplus s = y) \text{ ou } (x = y)$$

O objetivo é encontrar a string s , assim como no **Problema de Bernstein-Vazirani**.

Esse problema é bem restrito: f deve ter uma estrutura especial, fazendo com que poucas funções sejam capazes de cumprir a promessa. Por isso, resolvê-lo é uma

tarefa difícil para computadores clássicos. Além disso, o problema não foi feito para ter importância prática, apenas didática.

Podemos dividir o problema em dois casos principais:

- s é a string nula, $s = 0^n$. Nesse caso, podemos simplificar a expressão da promessa para $f(x) = f(y) \leftrightarrow x = y$, que é o mesmo que dizer que f é uma função “de um-para-um”, ou seja, $f(x)$ deve ser única para cada string x possível.
- $s \neq 0^n$. Nesse caso, f é “dois-para-um”: para cada output, existem dois inputs possíveis, de forma que $f(x) = f(x \otimes s)$.

A seguir está um exemplo de função $f: \Sigma^3 \rightarrow \Sigma^5$ que satisfaz a promessa para $s = 011$:

$$f(000) = 10011$$

$$f(001) = 00101$$

$$f(010) = 00101$$

$$f(011) = 10011$$

$$f(100) = 11010$$

$$f(101) = 00001$$

$$f(110) = 00001$$

$$f(111) = 11010$$

Há oito inputs e quatro outputs únicos; por inspeção, podemos ver que, para qualquer par de inputs que geram o mesmo resultado, vemos que a operação XOR bitwise entre os elementos do par é sempre 011.

Para ficar mais claro, repare nos dois casos a seguir:

$$f(000) = f(011) = 10011$$

$$f(101) = f(110) = 00001$$

$$000 \oplus 011 = 011$$

$$101 \oplus 110 = 011$$

Nenhum algoritmo clássico, mesmo que probabilístico, consegue encontrar a resposta em poucas queries. Os mais eficientes ainda possuem complexidade na ordem de $O(\sqrt{2^n})$, ou seja, crescem exponencialmente com o tamanho n da string.

O **Algoritmo de Simon**, com estrutura similar ao de **Deutsch-Jozsa**, consegue resolver o problema com complexidade linear, demonstrando novamente uma vantagem exponencial.

A figura a seguir ilustra o circuito que representa o Algoritmo de Simon, evidenciando suas similaridades com o Circuito de Deutsch-Jozsa.

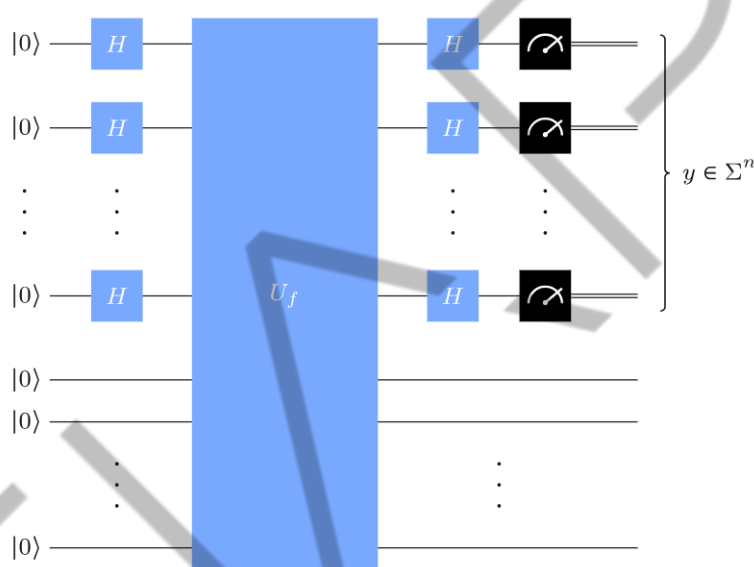


Figura 1 - Circuito de Simon
Fonte: IBM Quantum (2025)

Há $n + m$ qubits no circuito: aos n primeiros são aplicadas portas H e os m últimos vão diretamente para a porta U_f .

Como todos os qubits iniciam no estado $|0\rangle$, sem preparar nenhum no estado $|-\rangle$, **não ocorrerá** o fenômeno de **phase kickback** desta vez.

Então, assim como no circuito de Deutsch-Jozsa, aplicamos outra camada de Hadamards apenas nos n primeiros qubits e os medimos.

A ideia do algoritmo é que, a cada execução, teremos uma string y que fornece alguma informação sobre a string oculta s . Para resolver o problema, rodamos o circuito independentemente várias vezes, seguidas de uma etapa de pós-processamento clássico. Após essas execuções, teremos estatisticamente de informação suficiente para calcular s corretamente.

3.1.1 Análise do Algoritmo de Simon

O começo da análise é muito parecido com o algoritmo de Deutsch-Jozsa. Após a primeira camada de Hadamards nos n primeiros qubits, teremos o estado:

$$\frac{1}{\sqrt{2^n}} \sum_{x \in \Sigma^n} |0^m\rangle |x\rangle$$

Ao aplicar U_f , o output do oráculo é comparado aos m qubits inferiores com XOR, resultando em:

$$\frac{1}{\sqrt{2^n}} \sum_{x \in \Sigma^n} |f(x)\rangle |x\rangle$$

Após a segunda camada de Hadamards:

$$\frac{1}{\sqrt{2^n}} \sum_{x \in \Sigma^n} \sum_{y \in \Sigma^n} (-1)^{x \cdot y} |f(x)\rangle |y\rangle$$

NOTA: neste ponto, o conjunto de portas Hadamard aplicadas a cada um dos n primeiros qubits efetivamente realiza a **Transformada de Hadamard**. Essa operação é um análogo quântico da Transformada de Fourier e é fundamental para converter a informação, que estava na fase dos qubits após a consulta ao oráculo, em um resultado mensurável que revela propriedades da string secreta s .

A partir daqui a análise passa a divergir dos algoritmos estudados anteriormente. Estamos interessados na probabilidade de medir os resultados em cada string $y \in \Sigma^n$, calculada como:

$$p(y) = \left\| \frac{1}{2^n} \sum_{x \in \Sigma^n} (-1)^{x \cdot y} |f(x)\rangle \right\|^2$$

Para entender melhor, ajustamos a notação. O **alcance** (*range*) de uma função f é o conjunto contendo todas as strings de output:

$$\text{range}(f) = \{f(x) : x \in \Sigma^n\}$$

Para cada string $z \in \text{range}(f)$, escrevemos o conjunto com todos os inputs que resultam em z como output como a pré-imagem de $\{z\}$ sob f , isto é, o conjunto de

todos os elementos que f mapeia ao conjunto original. Denotamos essa pré-imagem como $f^{-1}(\{z\})$ (não confundir com a inversa de f).

$$f^{-1}(\{z\}) = \{x \in \Sigma^n : f(x) = z\}$$

Com essa notação, podemos separar a somatória na expressão de probabilidade acima para obter:

$$p(y) = \left\| \frac{1}{2^n} \sum_{z \in \text{range}(f)} \left(\sum_{x \in f^{-1}(z)} (-1)^{x \cdot y} |z\rangle \right) \right\|^2$$

Cada string $x \in \Sigma^n$ é representada apenas uma vez pelas somas, a expressão acima apenas separa em duas partes, dependendo em qual output $z = f(x)$ elas produzem ao computar f , e somando todas as partes ao final.

Calculando a norma euclidiana e simplificando a expressão, temos:

$$p(y) = \frac{1}{2^{2n}} \sum_{z \in \text{range}(f)} \left| \sum_{x \in f^{-1}(\{z\})} (-1)^{x \cdot y} \right|^2$$

Para simplificar mais, analisemos o módulo ao final da expressão para um $z \in \text{range}(f)$ arbitrário.

Se for uma string $s = 0^n$, só haverá um único elemento $x \in f^{-1}(\{z\})$, $\forall z \in \text{range}(f)$. Nesse caso, o resultado do módulo será 1.

Por outro lado, se $s \neq 0^n$, há exatamente duas strings no conjunto $f^{-1}(\{z\})$. Se escolhermos w para ser uma dessas strings, a outra será $w \oplus s$ devido à promessa do problema. Com essas observações (e ocultando alguns passos), podemos simplificar a expressão como:

$$\left| \sum_{x \in f^{-1}(\{z\})} (-1)^{x \cdot y} \right|^2 = |1 + (-1)^{y \cdot s}|^2 = \begin{cases} 4, & \text{se } y \cdot s = 0 \\ 0, & \text{se } y \cdot s = 1 \end{cases}$$

Em ambos os casos, podemos ver que o resultado da expressão não depende do z escolhido. Podemos finalizar a análise olhando novamente para cada caso, isoladamente:

- $s = 0^n$: há 2^n strings $z \in \text{range}(f)$, e temos:

$$p(y) = \frac{1}{2^{2n}} \cdot 2^n = \frac{1}{2^n}$$

Ou seja, medimos uma string $y \in \Sigma^n$ escolhida de forma uniforme e aleatória.

- $s \neq 0^n$: há 2^{n-1} elementos em $\text{range}(f)$, e temos:

$$p(y) = \frac{1}{2^{2n}} \sum_{z \in \text{range}(f)} \left| \sum_{x \in f^{-1}(\{z\})} (-1)^{x \cdot y} \right|^2 = \begin{cases} \frac{1}{2^{n-1}}, & \text{se } y \cdot s = 0 \\ 0, & \text{se } y \cdot s = 1 \end{cases}$$

Aqui, obtemos uma string escolhida de forma uniforme e aleatória a partir do conjunto $\{y \in \Sigma^n: y \cdot s = 0\}$, que contém 2^{n-1} elementos.

3.1.2 Camada de Pós-Processamento Clássico

Sabendo essas probabilidades, temos informação suficiente para determinar corretamente a string s , contanto que o processo seja repetido várias vezes e aceitando que teremos uma possibilidade de falha (que pode ser minimizada após executar o circuito algumas poucas vezes).

A ideia é que cada execução nos dá evidência estatística sobre s , que podemos utilizar para encontrar a string completa com altas probabilidades, executando o circuito vezes o bastante. Por exemplo, se executarmos o circuito k vezes, em que $k = n + 10$, teremos mais de 99.9% de chances de encontrar s corretamente.

Executando o circuito k vezes, teremos as strings $y^1, \dots, y^k \in \Sigma^n$ (lembrando que o valor acima de y é parte do nome, ou índice, da string, e não um expoente numérico de fato), de forma que:

$$y^1 = y^1_{n-1} \cdots y^1_0$$

$$y^2 = y^2_{n-1} \cdots y^2_0$$

⋮

$$y^k = y^k_{n-1} \cdots y^k_0$$

Formamos uma matriz $M_{k \times n}$ com os bits das strings como valores binários:

$$M = \begin{pmatrix} y_{n-1}^1 & \cdots & y_0^1 \\ y_{n-1}^2 & \cdots & y_0^2 \\ \vdots & \ddots & \vdots \\ y_{n-1}^k & \cdots & y_0^k \end{pmatrix}$$

Não sabemos s neste ponto, mas suponha que a conhecemos e formamos um vetor v com seus bits. Ao efetuar $Mv \bmod 2$, obtemos um **vetor nulo**:

$$Mv = \begin{pmatrix} y^1 \cdot s \\ y^2 \cdot s \\ \vdots \\ y^k \cdot s \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{pmatrix}$$

Ou seja, se tratarmos s como um vetor coluna v , ela sempre estará no espaço nulo de M (se calcularmos o módulo 2). Isso é verdade tanto para $s = 0^n$ quanto para $s \neq 0^n$.

Dessa forma, o vetor em que todos os elementos são zero sempre estará no espaço nulo de M , junto do vetor cujas entradas são os bits de s quando $s \neq 0^n$.

A questão é se existem outros vetores nesse espaço nulo; conforme k aumenta, isso fica cada vez mais improvável. No geral, se usarmos $k = n + r$, para algum inteiro positivo r qualquer, a chance de que os vetores correspondentes a 0^n e s sejam os únicos será de $1 - 2^{-r}$.

Utilizando álgebra linear, podemos calcular a descrição do espaço nulo de $M \bmod 2$. Contanto que os vetores 0^n e s estejam sozinhos no espaço nulo, podemos deduzir s a partir dos resultados do cálculo. Assim, podemos encontrar a resposta para o problema com altas probabilidades.

Apesar de não ter muitas aplicações práticas, a ponto de não fazer sentido escrever um código demonstrando o algoritmo, ele serviu para demonstrar conceitos que foram utilizados posteriormente em situações importantes.

3.2 O Algoritmo de Shor e o Fim da Era RSA

No Problema de Simon, temos um algoritmo quântico que demonstra uma vantagem exponencial sobre qualquer abordagem clássica para encontrar um

“período secreto”, que é a string s , em uma função f tal que $f(x) = f(y)$ se, e somente se, $y = x \oplus s$.

Peter Shor se inspirou e generalizou essa ideia fundamental: em vez de buscar um período sob a operação **XOR**, ele se propôs a encontrar o período de uma função sob a aritmética modular.

A grande ideia de Shor foi, portanto, reduzir o problema da fatoração a um problema de descoberta de período (order-finding), inspirado pelo algoritmo de Simon. Mas o problema da fatoração é justamente a parte difícil para computadores clássicos, que garante a segurança da criptografia RSA.

Um algoritmo que encontra o período de uma função de forma eficiente permite que qualquer pessoa com acesso à chave pública calcule a chave privada, quebrando a segurança do sistema RSA.

Para entender como o Algoritmo de Shor possibilita esse feito, precisamos relembrar duas ferramentas fundamentais que estudamos nos capítulos anteriores: a **Transformada Quântica de Fourier** (QFT) e uma de suas aplicações, a **Estimativa de Fase Quântica** (Quantum Phase Estimation).

A estimativa de fase é a técnica quântica que nos permite encontrar o período de uma função de forma eficiente e está no cerne do algoritmo de Shor.

Assim como o circuito de Simon utiliza a **Transformada de Hadamard** (um caso especial da QFT) para extrair o período s , Shor emprega a QFT para extrair o período r de uma função modular.

3.2.1 Encontrando Fatores Primos

Antes de construirmos o circuito quântico, é fundamental entender a redução matemática que transforma o problema de fatoração em um problema de busca de período. Esta etapa é puramente clássica, baseada em resultados da teoria dos números.

O objetivo é encontrar os fatores primos, p e q , de um número inteiro composto N . O procedimento clássico que serve de base para o Algoritmo de Shor é:

- **Escolher** um número a tal que $1 < a < N$.

- **Verificar** se a é coprimo a N , calculando $\text{mdc}(a, N)$;
 - Se o resultado for maior que 1, encontramos um fator não trivial de N e o problema está resolvido. Essa verificação pode ser feita de forma eficiente com o **algoritmo de Euclides** clássico.
 - Se $\text{mdc}(a, N) = 1$, o que é muito provável para N grandes, prosseguimos para o próximo passo.
- **Encontrar** o período é a parte mais difícil: o passo crucial é encontrar o período (ou ordem) r de a módulo N .
 - O período r é o menor inteiro positivo tal que $a^r \equiv 1 \pmod{N}$. Essa é a etapa computacionalmente intratável para um computador clássico, e é exatamente aqui que o algoritmo quântico entrará em ação.
- **Verificar** o período: uma vez que r seja encontrado, verificamos se ele é um número par.
 - Se for ímpar, o método falha para essa escolha de a , e devemos retornar ao primeiro passo com outro número aleatório.
- **Calcular** os fatores: se r for par, calculamos $x = a^{(r/2)}$. Se $x \equiv 1 \pmod{N}$, ou seja, $x + 1$ é múltiplo de N , o método também falha e devemos retornar ao passo 1. Caso contrário, temos a relação:

$$\begin{aligned} a^r &\equiv 1 \pmod{N} \rightarrow (a^{(r/2)})^2 - 1 \equiv 0 \pmod{N} \rightarrow \\ &\rightarrow (a^{(r/2)} - 1)(a^{(r/2)} + 1) \equiv 0 \pmod{N} \end{aligned}$$

Isso significa que N divide o produto $(a^{(r/2)} - 1)(a^{(r/2)} + 1)$. Como $a^{(r/2)}$ não é congruente a $\pm 1 \pmod{N}$, N não divide nenhum dos termos individualmente. Portanto, os fatores primos de N devem estar distribuídos entre esses dois termos.

- **Obter** os fatores finais: os fatores p e q de N podem ser encontrados calculando $\text{mdc}(a^{(r/2)} - 1, N)$ e $\text{mdc}(a^{(r/2)} + 1, N)$.
 - Pode-se provar que, para uma escolha aleatória de a , a probabilidade de que r seja par e $a^{(r/2)} \not\equiv -1 \pmod{N}$ é de, pelo menos, 50%.

Esta redução matemática mostra que o ponto central do problema de fatoração reside na capacidade de encontrar o período r de forma eficiente. A seguir, veremos

como um circuito quântico, ao aplicar os princípios de superposição e interferência por meio da Estimativa de Fase Quântica, resolve exatamente esse desafio.

3.2.2 Circuito Quântico para Descoberta de Período

Durante o passo a passo anterior, vimos como o problema de fatorar N pode ser classicamente reduzido à tarefa de encontrar o período r de $a \pmod{N}$. A maior parte do procedimento pode ser executado por computadores clássicos, porém, determinar r é a etapa computacionalmente intratável, e é aqui que a Computação Quântica oferece uma solução eficiente.

O problema de encontrar o período (ou ordem) de um elemento em um grupo é um problema matemático bem conhecido, chamado de **order-finding**. Como vimos, um algoritmo clássico para resolver o problema de fatoração se torna eficiente se tiver acesso a uma sub-rotina que resolve o order-finding de forma rápida. A grande contribuição de Peter Shor foi demonstrar que o problema de order-finding pode ser mapeado para o problema de **estimativa de fase quântica (phase estimation)**, que pode ser resolvido eficientemente por um computador quântico.

O circuito quântico do Algoritmo de Shor é, na verdade, uma aplicação direta da **Estimativa de Fase Quântica (QPE)**, uma das principais aplicações da **QFT** que estudamos anteriormente. A QPE é projetada para estimar o autovalor de um operador unitário. Dado um operador U e um autovetor $|u\rangle$ tal que $U|u\rangle = e^{2\pi i\phi}|u\rangle$, a QPE nos permite encontrar uma boa aproximação da fase ϕ . A conexão com a descoberta de período se torna clara quando definimos o operador unitário U para realizar a multiplicação modular por a :

$$U|y\rangle = |ay \pmod{N}\rangle$$

Os autovetores $|u_s\rangle$ deste operador são dados por:

$$|u_s\rangle \equiv \frac{1}{r} \sum_{k=0}^{r-1} e^{-2\pi i s k / r} |a^k \pmod{N}\rangle$$

E seus autovalores correspondentes são $e^{2\pi is/r}$, em que r é o período que estamos buscando. Portanto, a fase de cada autovetor codifica uma fração $\frac{s}{t}$. É essa fase que o circuito quântico irá estimar.

A estrutura do circuito é idêntica à do algoritmo de QPE, ilustrado na figura “Primeira parte do Circuito QPE”, e utiliza dois registradores de qubits:

- **Registrador de Contagem (superior):** composto por t qubits, onde t é escolhido para garantir precisão suficiente (tipicamente $t \approx 2L$, sendo L o número de bits de N). Este registrador armazenará a estimativa da fase.
- **Registrador Alvo (inferior):** composto por L qubits, suficiente para armazenar o número N . Esse registrador conterá o autovetor $|u\rangle$ sobre o qual o operador U atua.

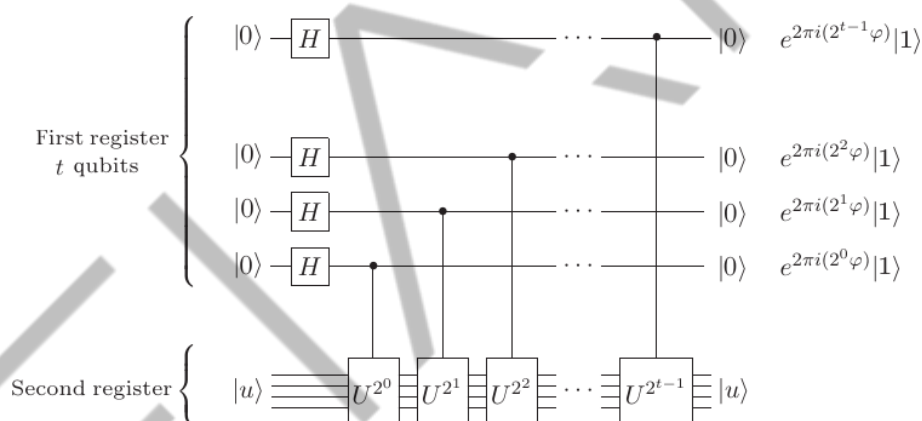


Figura 2 - Primeira parte do Circuito QPE
Fonte: Nielsen & Chuang (2010)

O circuito executa os seguintes passos de forma sequencial:

Etapa 1: Preparação do Estado Inicial

Começamos com ambos os registradores no estado $|00 \dots 0\rangle$. Em seguida, preparamos o estado de cada registrador para a computação:

- No registrador de contagem, aplicamos uma porta de Hadamard em cada um dos t qubits. Essa operação, conhecida como **Transformada de Hadamard** ($H^{\otimes t}$), cria uma superposição uniforme de todos os 2^t estados possíveis.

- O registrador alvo é inicializado no estado $|1\rangle$. Matematicamente, o estado $|1\rangle$ pode ser expresso como uma superposição de todos os autovetores $|u_s\rangle$, o que garante que a estimativa de fase funcione para todos os possíveis valores de $\frac{s}{r}$.
 - No QPE padrão, o estado deste registrador é inicializado como $|u\rangle$. Porém, as propriedades do problema de order-finding nos permite simplificar o estado para $|1\rangle$.

O estado inicial combinado do sistema, $|\psi_1\rangle$, é o produto tensorial dos estados dos dois registradores:

$$|\psi_1\rangle = \frac{1}{\sqrt{2^t}} \sum_{j=0}^{2^t-1} |j\rangle |1\rangle$$

Etapa 2: Exponenciação Modular Controlada

Esta é a etapa central do algoritmo, na qual a função periódica é computada e sua periodicidade é “impressa” na fase do registrador de contagem. Para cada qubit k (de 0 a $t-1$) no registrador de contagem, aplicamos uma operação U controlada no registrador-alvo.

O estado do sistema evolui para:

$$|\psi_2\rangle = \frac{1}{\sqrt{2^t}} \sum_{j=0}^{2^t-1} |j\rangle U^j |1\rangle$$

Como o estado $|1\rangle$ do registrador-alvo é uma superposição dos autovetores de U , a fase s/r de cada autovetor é transferida de volta para o registrador de contagem através do fenômeno de **phase kickback**. Isso entrelaça os dois registradores, e o estado pode ser aproximado por:

$$|\psi_2\rangle \approx \frac{1}{\sqrt{r 2^t}} \sum_{s=0}^{r-1} \sum_{j=0}^{2^t-1} e^{2\pi i s j / r} |j\rangle |u_s\rangle$$

Neste ponto, a informação sobre o período r está codificada na fase relativa dos estados do primeiro registrador.

Etapa 3: Transformada Quântica de Fourier Inversa (QFT^{-1})

Para extrair a informação da fase, aplicamos a **Transformada Quântica de Fourier Inversa** no registrador de contagem. Essa transformação utiliza a interferência quântica para converter a fase que foi codificada em amplitudes de probabilidade mensuráveis.

O estado do sistema se torna:

$$|\psi_3\rangle = \frac{1}{\sqrt{r}} \sum_{s=0}^{r-1} |\widetilde{s/r}\rangle |u_s\rangle = \frac{1}{\sqrt{r}} \sum_{s=0}^{r-1} |\sim(s/r)\rangle |u_s\rangle$$

Em que $|\sim(s/r)\rangle$ é um estado que representa a melhor **aproximação inteira** de $\frac{2^t \cdot s}{r}$ para um valor aleatório de s entre 0 e $r - 1$.

Etapa 4: Medição

Finalmente, realizamos uma medição no registrador de contagem, e o resultado será um valor inteiro y . O modelo do circuito completo para o problema de order-finding, com as devidas abstrações, está ilustrado na figura:

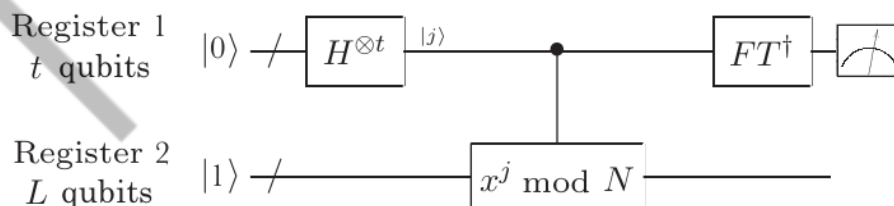


Figura 3 - Circuito do Algoritmo de Order-Finding
Fonte: Nielsen & Chuang (2010)

NOTA: os autores utilizaram uma notação um pouco diferente: o registrador superior compacta os t primeiros qubits, enquanto o inferior compacta os L seguintes.

Além disso, Nielsen e Chuang utilizam a notação $FT^\dagger$ para a Transformada Quântica de Fourier Inversa, ao contrário da $QFT^\dagger$ normalmente utilizada pelo Qiskit e IBM.

Por fim, a porta indicando $x^j \bmod N$ é equivalente (no algoritmo de order-finding) à U^j descrita no procedimento de QPE, visto que x é o nosso a .

Com alta probabilidade, este valor y satisfaz a seguinte relação para algum inteiro aleatório s :

$$\frac{y}{2^t} \approx \frac{s}{r}$$

Com este valor y em mãos, o desafio se torna novamente clássico. A seguir, veremos como extrair o período r a partir desta aproximação.

3.2.3 Pós-processamento Clássico

Precisamos agora utilizar essa aproximação para determinar os inteiros s e r . A ferramenta matemática ideal para essa tarefa é o algoritmo de frações contínuas. Este é um método clássico e eficiente para encontrar a melhor aproximação racional para um número real.

A teoria garante que, se a aproximação $\frac{y}{2^t}$ estiver suficientemente próxima de $\frac{s}{r}$, então $\frac{s}{r}$ será um dos convergentes gerados pelo algoritmo de frações contínuas. O número de qubits t no registrador de contagem é escolhido justamente para garantir que essa condição seja satisfeita.

O procedimento clássico de pós-processamento, portanto, é o seguinte:

- **Calcular** a fração decimal $\frac{y}{2^t}$ a partir do resultado da medição y e do tamanho do registrador t .
- **Aplicar** o algoritmo de frações contínuas a $\frac{y}{2^t}$ para obter uma lista de **frações candidatas** $\frac{s'}{r'}$.

- **Testar** cada candidato: para cada fração $\frac{s'}{r'}$ obtida, o denominador r' é um **candidato** para o período r . Verificamos se ele é, de fato, o **período verdadeiro**, testando se $a^{r'} \equiv 1 \pmod{N}$.
 - Se um r' válido for encontrado, prosseguimos com os passos finais do Algoritmo de Shor, que também são clássicos (verificar se r' é par e demais etapas descritas na seção “Encontrando Fatores Primos”).
 - Se o teste falhar ou se nenhum dos convergentes funcionar, o algoritmo quântico falhou para esta medição e deve ser executado novamente.

Com o período r em mãos, a quebra da criptografia RSA se torna uma tarefa computacionalmente **tratável** em um **computador clássico**.

3.2.4 Algoritmo de Shor com Qiskit

Vamos implementar o Algoritmo de Shor em Qiskit para um exemplo concreto, fatorando o número $N = 15$. Este exemplo é pequeno o suficiente para ser simulado, mas complexo o bastante para ilustrar todos os passos do algoritmo.

Retomaremos um cenário similar ao da seção “Exemplo Prático”, supondo uma situação na qual Eve intercepta a chave pública de Bob e descobre seu N . Embora lá o valor fosse $N = 55$, para a simulação usaremos $N = 15$, que é computacionalmente tratável em qualquer simulador clássico.

O Qiskit já possui uma implementação otimizada do algoritmo, porém, para fins didáticos e para facilitar a compatibilidade entre diferentes versões, construiremos o circuito completo, que pode ser executado em simuladores ou, futuramente, em computadores quânticos reais.

O código será estruturado para refletir as etapas teóricas: preparação dos qubits, aplicação do oráculo quântico (exponenciação modular), QFT inversa e medição.

Além disso, como construir um circuito genérico e eficiente para o algoritmo de Shor é uma tarefa extremamente complexa e extensa, novamente fugindo do escopo do curso (além de não haver consenso sobre o melhor circuito possível), nosso circuito será construído pensando no exemplo em questão, no qual $N = 15$ e $a = 7$.

NOTA: os códigos a seguir serão construídos de forma sequencial, partindo do pressuposto que os anteriores estão disponíveis no arquivo. Como sempre, recomenda-se o uso de ambientes similares ao Jupyter Notebook.

O código-fonte a seguir importa as bibliotecas necessárias e define uma função clássica para calcular o Máximo Divisor Comum com o algoritmo de Euclides, que será utilizada no pós-processamento.

```
import numpy as np
import math
from fractions import Fraction

from qiskit import QuantumCircuit, QuantumRegister,
ClassicalRegister
from qiskit.circuit.library import QFT
from qiskit_aer.primitives import Sampler

def mdc(a, b):
    """Calcula o Máximo Divisor Comum usando o algoritmo
    de Euclides."""
    while b:
        a, b = b, a % b
    return a
```

Código-fonte 1 - Algoritmo de Shor: MDC
Fonte: Elaborado pelo autor (2025)

O núcleo do circuito é a exponenciação modular controlada. Esta é uma das etapas em que construir um circuito genérico é um desafio complexo de aritmética quântica. Para o nosso exemplo, a operação de multiplicação modular é implementada com portas *SWAP*, como fazemos no código-fonte a seguir.

```
class ModularExponentiation(QuantumCircuit):
    """
    Cria o circuito para a exponenciação modular  $a^x \bmod N$ .

    Esta é uma implementação simplificada que usa portas
    controladas
    para cada potência de  $a$ . Para  $N=15$ , isso é viável.
    """
    def __init__(self, n_contagem, n_alvo, a, N):
        super().__init__(n_contagem + n_alvo,
        name=f"U({a}^x mod {N})")

        # Registradores para facilitar a referência
        q_contagem = self.qubits[:n_contagem]
```

```

        q_alvo = self.qubits[n_contagem:]

        for i in range(n_contagem):
            # Calcula  $a^{(2^i)} \bmod N$ 
            exp_a = pow(a, 2**i, N)

            U_a = self.create_controlled_Ua(exp_a,
q_alvo, N)
            self.append(U_a.control(1), [q_contagem[i]]
+ q_alvo)

        def create_controlled_Ua(self, a, q_alvo, N):
            """Cria a porta que aplica a multiplicação por
'a' mod N."""
            U = QuantumCircuit(len(q_alvo), name=f"U({a})")
            n = len(q_alvo)

            # Este é um exemplo específico para N=15 e a=7.
            # Uma implementação geral é muito mais complexa.
            # Para a=7, N=15:
            #  $7^1 \bmod 15$ :  $|x\rangle \rightarrow |7x \bmod 15\rangle$  SWAP(0,1),
SWAP(1,2), SWAP(2,3)
            #  $7^2 \bmod 15$ :  $|x\rangle \rightarrow |4x \bmod 15\rangle$  SWAP(0,2),
SWAP(1,3)
            #  $7^4 \bmod 15$ :  $|x\rangle \rightarrow |1x \bmod 15\rangle$  (Identidade)
            if a == 7:
                U.swap(0, 1); U.swap(1, 2); U.swap(2, 3)
            elif a == 4:
                U.swap(0, 2); U.swap(1, 3)
            elif a == 13:
                U.swap(1, 2); U.swap(0, 1)
                U.x(0); U.x(1); U.x(2); U.x(3)
            elif a == 11:
                U.swap(0,1); U.swap(2,3); U.swap(0,2)
            # Para a=1 (identidade), não fazemos nada.

            return U
    
```

Código-fonte 2 - Algoritmo de Shor: Oráculo
 Fonte: Elaborado pelo autor (2025)

Com o oráculo definido, montamos o circuito completo. O número de qubits de contagem ($n_{contagem}$) é tipicamente o dobro do número de bits de N para garantir a precisão da Estimativa de Fase. O processo, definido no código-fonte “**Algoritmo de Shor: Circuito**”, segue os passos da QPE:

- **Inicialização:** aplicamos uma Transformada de Hadamard nos qubits de contagem e inicializamos o registrador alvo em $|1\rangle$

- **Exponenciação Modular Controlada:** aplicamos o oráculo customizado.
- **QFT Inversa e Medição:** aplicamos a Transformada Quântica de Fourier Inversa no registrador de contagem e o medimos.

```
def criar_circuito_shor(a, N):  
    """  
    Constrói o circuito quântico completo para o algoritmo  
de Shor.  
    """  
    n = N.bit_length()  
    n_contagem = 2 * n  
    qr_contagem = QuantumRegister(n_contagem,  
name="contagem")  
    qr_alvo = QuantumRegister(n, name="alvo")  
    cr_resultado = ClassicalRegister(n_contagem,  
name="resultado")  
  
    circuito = QuantumCircuit(qr_contagem, qr_alvo,  
cr_resultado)  
  
    # 1. Superposição  
circuito.h(qr_contagem)  
  
    # 2. Inicializar alvo em |1>  
circuito.x(qr_alvo[0])  
circuito.barrier()  
  
    # 3. Aplicar a exponenciação modular controlada  
exp_mod_gate = ModularExponentiation(n_contagem, n,  
a, N)  
circuito.append(exp_mod_gate, qr_contagem[:] +  
qr_alvo[:])  
circuito.barrier()  
  
    # 4. QFT Inversa  
iqft_gate = QFT(num_qubits=n_contagem, inverse=True,  
name="IQFT")  
circuito.append(iqft_gate, qr_contagem)  
circuito.barrier()  
  
    # 5. Medição  
circuito.measure(qr_contagem, cr_resultado)  
  
    return circuito, n_contagem
```

Código-fonte 3 - Algoritmo de Shor: Circuito
Fonte: Elaborado pelo autor (2025)

Vamos executar o circuito no *Sampler* do Qiskit Aer, que retorna uma distribuição de probabilidades sobre os possíveis resultados da medição. A medição mais provável, y , nos dá a estimativa da fase $\frac{s}{r}$.

Utilizamos então o algoritmo de frações contínuas (implementado pela biblioteca *fractions* do Python) para extrair o período r a partir da fase. Com r em mãos, calculamos os fatores p e q classicamente. Essa camada de pós-processamento clássico, que será executada após a medição, está descrita no código-fonte a seguir.

```
def post_processar_resultados(job_result, n_contagem, N,
a):
    """
    Processa os resultados da medição para encontrar os
    fatores.
    """
    quasi_dist = job_result.quasi_dists[0]

    print(f"\nResultados da medição (probabilidades >
0.01):")
    # Mostra os resultados mais prováveis para análise
    for shot, prob in quasi_dist.binary_probabilities().items():
        print(f"    - Medida: {shot} (decimal: {int(shot,
2)}), Probabilidade: {prob:.4f}")

    # Pega o resultado mais provável
    y = max(quasi_dist, key=quasi_dist.get)
    print(f"\nMedida mais provável: y = {y}")

    fase = y / (2*n_contagem)
    print(f"Fase estimada:  $\phi = \{y\}/\{2*n\_contagem\} =
\{fase:.5f\}")

    # Algoritmo de frações contínuas
    frac = Fraction(fase).limit_denominator(N)
    r = frac.denominator
    print(f"Fração          contínua          s/r:
{frac.numerator}/{frac.denominator} -> Período (r) candidato:
{r}")

    if r % 2 != 0:
        print(f"O período r={r} é ímpar. O algoritmo
falhou para este 'a'. Tente outro 'a'.")
        return None, None

    # Calcular os fatores$ 
```

```

        try:
            fator1 = mdc(pow(a, r // 2) + 1, N)
            fator2 = mdc(pow(a, r // 2) - 1, N)

            if fator1 == N or fator2 == N or fator1 == 1 or
fator2 == 1:
                print("Encontrados fatores triviais. O
algoritmo falhou para este 'a'.")
                return None, None

            return fator1, fator2
        except ValueError:
            print("Erro no cálculo dos fatores
(potencialmente r=0).")
            return None, None
    
```

Código-fonte 4 - Algoritmo de Shor: Pós-processamento
 Fonte: Elaborado pelo autor (2025)

Por fim, criamos a função principal do exemplo, que define o N e a e executa o algoritmo, como mostra o próximo código-fonte.

Adicionamos logs mostrando os resultados do passo a passo sendo processado, para facilitar análises e indicar em qual momento o algoritmo falhou, caso aconteça algum problema.

```

N = 15
# Escolha de 'a'. a=7 é uma boa escolha para N=15, pois
tem período 4.
# Outras boas escolhas são 11, 13. 'a' não pode ter fatores
em comum com N.
a = 7

print(f"--- Algoritmo de Shor para Fatorar N = {N} ---")

# --- Parte 1: Checagem Clássica ---
if mdc(a, N) != 1:
    print(f"Sorte! O mdc(a, N) não é 1. Um fator é {mdc(a,
N)}.")
else:
    print(f"Escolhido a = {a}. mdc(a, N) = 1. Prosseguindo
para a parte quântica.")

# --- Parte 2: Construção e Simulação do Circuito
Quântico ---
circuito_shor, n_contagem = criar_circuito_shor(a, N)

# Opcional: Desenhar o circuito
# print("\nCircuito Quântico de Shor (N=15, a=7):")
# display(circuito_shor.draw(output='mpl', fold=-1))
    
```

```

print("\nIniciando a simulação com o Sampler do Qiskit
Aer...")
sampler = Sampler()
job = sampler.run(circuito_shor, shots=1024)
result = job.result()

# --- Parte 3: Pós-processamento ---
fator1, fator2 = post_processar_resultados(result,
n_contagem, N, a)

if fator1 and fator2:
    print("\n--- SUCESSO! ---")
    print(f"Os fatores de {N} são: {fator1} e
{fator2}")
    print(f"Verificação: {fator1} * {fator2} =
{fator1 * fator2}")
else:
    print("\n--- FALHA ---")
    print("Não foi possível encontrar os fatores com
a medição obtida. Tente executar novamente ou escolher outro
'a'.")

```

Código-fonte 5 - Algoritmo de Shor: Execução

Fonte: Elaborado pelo autor (2025)

Durante a execução do código, caso não ocorra falhas, teremos nas últimas linhas dos logs os fatores de N e a verificação, multiplicando-os para garantir que o algoritmo encontrou os fatores corretamente.

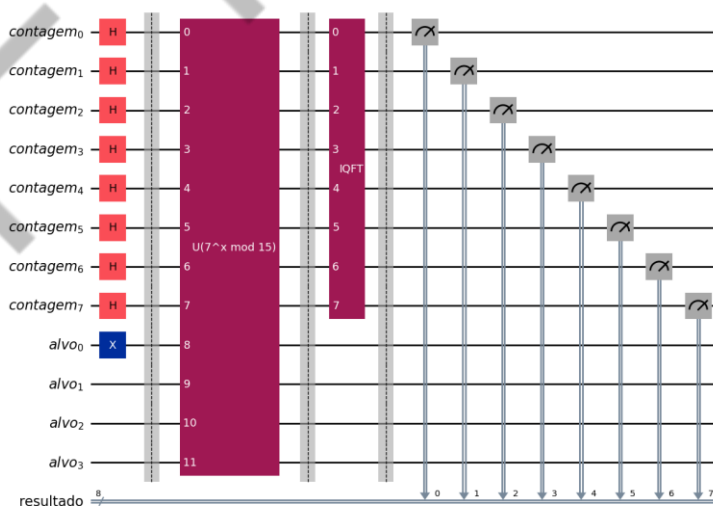


Figura 4 - Circuito do Algoritmo de Shor

Fonte: Elaborado pelo autor (2025)

A figura anterior ilustra o circuito criado durante a execução do algoritmo, enquanto a figura “Resultados do Algoritmo de Shor” mostra os logs de uma execução que obteve os fatores de N corretamente.

```
Iniciando a simulação com o Sampler do Qiskit Aer...

Resultados da medição (probabilidades > 0.01):
- Medida: 01000000 (decimal: 64), Probabilidade: 0.2734
- Medida: 00000000 (decimal: 0), Probabilidade: 0.2324
- Medida: 10000000 (decimal: 128), Probabilidade: 0.2627
- Medida: 11000000 (decimal: 192), Probabilidade: 0.2314

Medida mais provável: y = 64
Fase estimada:  $\varphi = 64/256 = 0.25000$ 
Fração contínua s/r: 1/4 → Período (r) candidato: 4

— SUCESSO! —
Os fatores de 15 são: 5 e 3
Verificação:  $5 * 3 = 15$ 
```

Figura 5 - Resultados do Algoritmo de Shor
Fonte: Elaborado pelo autor (2025)

O exemplo ilustra como a combinação de uma sub-rotina quântica com pós-processamento clássico quebra a fatoração de inteiros, o pilar do criptossistema RSA.

4 INTRODUÇÃO À CRIPTOGRAFIA QUÂNTICA

Até o momento, focamos em como a Computação Quântica representa uma ameaça existencial para a criptografia de chave pública moderna. Algoritmos como o de Shor demonstram que a segurança baseada na dificuldade computacional de problemas como a fatoração de inteiros pode ser quebrada por um computador quântico de larga escala, tornando vulneráveis sistemas como o RSA.

No entanto, a mesma mecânica quântica que cria a ameaça também oferece a solução. Este é o outro lado da “faca de dois gumes” da Computação Quântica na segurança da informação: a promessa de métodos de comunicação cuja segurança não se baseia em suposições sobre a dificuldade de problemas matemáticos, mas nas próprias leis fundamentais da física.

O campo que explora essa possibilidade é a **Criptografia Quântica**. É importante distinguir este termo amplo da sua aplicação mais proeminente e prática: a **Distribuição de Chaves Quânticas** (QKD, Quantum Key Distribution).

A QKD não é um método para criptografar a mensagem em si. Em vez disso, é um protocolo que permite as duas partes, Alice e Bob, gerar e compartilhar uma chave secreta perfeitamente aleatória.

Uma vez que ambos possuam essa chave, eles podem usá-la em um criptosistema simétrico comprovadamente seguro, como o one-time pad, para se comunicarem com sigilo absoluto.

A genialidade da QKD reside no fato de que qualquer tentativa de uma parte espia, Eve, de interceptar a chave durante sua criação será detectada.

A seguir, exploraremos como esse novo paradigma de segurança funciona, com foco no protocolo mais famoso e fundamental da área: o **protocolo BB84**.

4.1 Novo Paradigma de Segurança

A segurança dos protocolos de QKD, como o BB84, baseia-se em dois princípios fundamentais da mecânica quântica, que garantem que qualquer tentativa de espionagem seja detectável por Alice e Bob.

O primeiro princípio é o fato de que **a medição perturba um sistema quântico**. Diferentemente dos sistemas clássicos, em que se pode “olhar” para um bit sem alterá-lo, a medição de um qubit em um estado desconhecido inevitavelmente corre o risco de colapsar sua função de onda e alterar seu estado.

Se Alice envia um qubit e Eve o intercepta para medi-lo, Eve é forçada a escolher uma base de medição. A menos que ela adivinhe corretamente a base em que o qubit foi preparado, sua medição alterará o estado do qubit que ela reenviará para Bob. Essa alteração introduz erros na sequência de bits que Bob registra, que podem ser detectados.

O segundo princípio, intrinsecamente ligado ao primeiro, é o **Teorema da Não-Clonagem**. Este teorema afirma que é impossível criar uma cópia idêntica e independente de um estado quântico arbitrário e desconhecido.

Isso frustra a estratégia mais óbvia de Eve: interceptar o qubit de Alice, criar uma cópia perfeita para si, e enviar o original intacto para Bob. Como a clonagem é proibida (impossível) pelas leis da física, Eve não consegue obter informação sobre o estado do qubit sem realizar uma medição, o que nos leva de volta ao primeiro princípio: a medição introduz perturbações detectáveis.

Juntos, esses dois pilares formam um novo paradigma de segurança. Alice e Bob podem utilizar um canal público para comparar uma amostra de sua chave gerada. Se a **taxa de erro quântico** (QBER, Quantum Bit Error Rate) for desprezível, eles podem ter alta confiança de que ninguém interceptou sua comunicação.

No entanto, se a taxa de erro for superior a um certo limiar, eles concluem que a linha não é segura, descartam a chave e tentam novamente. A segurança, portanto, passa de uma esperança baseada na limitação tecnológica do adversário para uma garantia baseada nas leis imutáveis da natureza.

4.2 Protocolo BB84

O **protocolo BB84** (nomeado em homenagem a seus criadores, Charles H. Bennett e Gilles Brassard, e ao ano de sua publicação, 1984) é o protocolo de Distribuição de Chaves Quânticas (QKD) mais fundamental e conhecido.

Ele utiliza os princípios quânticos que acabamos de discutir para permitir que Alice e Bob estabeleçam uma chave secreta compartilhada através de um canal público e um canal quântico.

A segurança do protocolo reside no fato de que qualquer tentativa de espionagem inevitavelmente introduz perturbações detectáveis. A comunicação quântica é realizada com sistemas quânticos individuais, como fótons polarizados.

O protocolo pode ser dividido em cinco etapas principais, envolvendo tanto o canal quântico quanto um canal público clássico para reconciliação.

4.2.1 Etapa 1: Envio (Alice)

O processo começa com Alice, a remetente, preparando e enviando uma sequência de qubits para Bob, o destinatário.

Cada qubit codifica um bit clássico, 0 ou 1. A relevância do protocolo está na forma como essa codificação é feita. Para cada bit que deseja enviar, Alice escolhe aleatoriamente uma de duas bases de medição, que são mutuamente conjugadas.

As bases mais comuns são:

Base Retilínea (+):

- O bit 0 é codificado no estado $|0\rangle$ (**polarização vertical**).
- O bit 1 é codificado no estado $|1\rangle$ (**polarização horizontal**).

Base Diagonal (×):

- O bit 0 é codificado no estado $|+\rangle$ (**polarização a +45°**).
- O bit 1 é codificado no estado $|-\rangle$ (**polarização a -45°**).

Essas duas bases são conjugadas: uma medição na base errada produz um resultado completamente aleatório.

Por exemplo, se Alice envia um qubit no estado $|+\rangle$ (bit 0 na base ×) e Bob mede na base retilínea (+), ele obterá os resultados 0 ou 1 com 50% de probabilidade cada.

O procedimento de Alice é o seguinte:

- **Gerar uma sequência de bits aleatórios:** esta será a base para a chave secreta. Exemplo: 10110010.
- **Gerar uma sequência de bases aleatórias:** para cada bit, ela escolhe aleatoriamente entre a base + e a base ×. Exemplo: + × + × × + + ×.
- **Codificar e Enviar:** Alice prepara um qubit para cada par bit-base e o envia a Bob pelo canal quântico.

A tabela a seguir ilustra um exemplo do que Alice envia.

Bits	1	0	1	1	0	0	1	0
Bases	+	×	+	×	×	+	+	×
Qubit Enviado	$ 1\rangle$	$ +\rangle$	$ 1\rangle$	$ -\rangle$	$ +\rangle$	$ 0\rangle$	$ 1\rangle$	$ +\rangle$

Tabela 1 - Exemplo de qubits enviados por Alice
Fonte: Elaborado pelo autor (2025)

É crucial que ambas as sequências (bits e bases) sejam genuinamente aleatórias.

Alice mantém suas duas sequências **em segredo** nesta etapa. Bob, por sua vez, se prepara para receber os qubits e realizar suas próprias medições.

4.2.2 Etapa 2: Medição (Bob)

Agora, a sequência de qubits viaja de Alice para Bob por meio do canal quântico. Bob não tem conhecimento prévio nem dos bits nem das bases que Alice utilizou.

Sua tarefa é medir cada qubit recebido para extrair a informação. Assim como Alice, para cada qubit, Bob escolhe aleatoriamente uma das duas bases de medição (a retilínea ou a diagonal).

O procedimento de Bob é o seguinte:

- **Gerar uma sequência de bases aleatórias:** para cada qubit que espera receber, Bob gera sua própria sequência aleatória de bases (+) e (×), independentemente de Alice.

- **Medir cada qubit:** ao receber cada qubit de Alice, ele o mede na base correspondente de sua sequência. O resultado de cada medição (0 ou 1) é registrado.

A física quântica dita o resultado desta etapa:

- Se Bob escolher a **mesma base** que Alice usou para preparar um qubit, sua medição revelará o bit original de Alice com **100% de certeza**.
 - Por exemplo, se Alice enviou $|1\rangle$ (bit 1, base +) e Bob mediu na base +, ele obterá o resultado 1 com certeza.
- Se Bob escolher a base errada (diferente da de Alice), o resultado de sua medição será completamente aleatório. A precisão, neste caso, cai para 50%.
 - Por exemplo, se Alice enviou $|+\rangle$ (bit 0, base $\times$) e Bob mediu na base +, ele obterá os resultados 0 ou 1, cada um com 50% de probabilidade.

A tabela a seguir dá continuidade ao nosso exemplo, mostrando as escolhas de Bob e os resultados obtidos. As medições em que as bases coincidiram estão destacadas em negrito.

Bits de Alice	1	0	1	1	0	0	1	0
Bases de Alice	+	$\times$	+	$\times$	$\times$	+	+	$\times$
Qubit Enviado	$ 1\rangle$	$ +\rangle$	$ 1\rangle$	$ -\rangle$	$ +\rangle$	$ 0\rangle$	$ 1\rangle$	$ +\rangle$
Bases de Bob	+	+	$\times$	$\times$	+	+	$\times$	$\times$
Bits de Bob	1	1	0	1	1	0	0	0

Tabela 2 - Exemplo de bits medidos por Bob
Fonte: Elaborado pelo autor (2025)

Neste ponto, nem Alice nem Bob sabem quais de seus bits correspondem. Eles resolverão isso na próxima etapa, utilizando um canal de comunicação clássico e público.

4.2.3 Etapa 3: Sifting

Neste ponto, tanto Alice quanto Bob possuem sequências de bits. No entanto, devido às medições em bases diferentes, a sequência de Bob é, em média, apenas 50% igual à de Alice e contém resultados aleatórios onde as bases não coincidiram.

Eles precisam de um método para descartar esses resultados, mantendo apenas os bits que deveriam ser idênticos. Este processo é conhecido como **sifting** (**peneiramento** ou **filtragem**) e é realizado por meio de um canal de comunicação clássico e público, de acordo com o procedimento a seguir:

- **Comunicação Pública:** para cada qubit enviado, Alice e Bob anunciam publicamente a base que utilizaram para a preparação e medição, respectivamente. É crucial que eles revelem apenas as bases, e não os valores dos bits que enviaram ou mediram.
- **Comparação e Descarte:** eles comparam as sequências de bases. Nos casos em que a base de Alice e a de Bob são diferentes, eles descartam o bit correspondente de suas sequências. Estatisticamente, isso ocorre em cerca de 50% dos casos.
- **Sifted Key:** os bits que restaram, onde as bases coincidiram, formam a chave filtrada (*sifted key*). Na ausência de ruído ou espionagem, essa chave deve ser idêntica para Alice e Bob.

A tabela a seguir finaliza o exemplo, ilustrando o processo de filtragem. As colunas em que as bases coincidem estão destacadas em negrito.

Bits de Alice	1	0	1	1	0	0	1	0
Bases de Alice	+	×	+	×	×	+	+	×
Qubit Enviado	$ 1\rangle$	$ +\rangle$	$ 1\rangle$	$ -\rangle$	$ +\rangle$	$ 0\rangle$	$ 1\rangle$	$ +\rangle$
Bases de Bob	+	+	×	×	+	+	×	×
Bits de Bob	1	1	0	1	1	0	0	0
Bases Iguais?	Sim	Não	Não	Sim	Não	Sim	Não	Sim
Sifted Key	1			1		0		0

Tabela 3 - Exemplo de Sifted Key
Fonte: Elaborado pelo autor (2025)

Após esta etapa, Alice e Bob compartilham a chave peneirada (1100, no exemplo). No entanto, eles ainda não podem considerá-la segura. A presença de uma espiã, Eve, teria introduzido erros nesta chave, e eles precisam de uma forma de detectar essa espionagem antes de usá-la para comunicação.

4.2.4 Etapa 4: Detecção de Espionagem

Neste ponto do protocolo, Alice e Bob precisam garantir que Eve não interceptou os qubits durante a transmissão.

Esta etapa explora o princípio fundamental de que a medição perturba um sistema quântico. Se Eve tentou espionar o canal quântico, ela foi forçada a realizar medições nos qubits enviados por Alice.

Como Eve não sabia as bases corretas que Alice usou, suas medições inevitavelmente introduziram erros na sequência de bits que Bob recebeu. Alice e Bob podem agora detectar esses erros, com o seguinte procedimento:

- **Comunicação Pública:** Alice e Bob selecionam um subconjunto aleatório de sua chave peneirada e comparam publicamente os valores desses bits. Este subconjunto serve como amostra de teste.
- **Cálculo da Taxa de Erro:** eles calculam a **Taxa de Erro Quântico de Bits** (QBER, Quantum Bit Error Rate), que é a porcentagem de bits na amostra que não coincidiram.

- **Decisão de Segurança:**

- Se a QBER for zero ou estiver abaixo de um limiar de segurança predefinido, Alice e Bob podem concluir, com alta confiança, que não houve espionagem. Eles prosseguem para a próxima etapa, descartando os bits que foram anunciados publicamente.
- Se a QBER for maior que o limiar, eles assumem que Eve está presente no canal. Nesse caso, toda a chave é considerada comprometida e é descartada. Eles devem então reiniciar todo o protocolo, possivelmente por um canal quântico diferente.

Continuando o exemplo, a chave peneirada de Alice e Bob é **1100**. Suponha que eles decidam usar o último bit (bit mais à direita) como amostra de teste.

Ambos anunciam publicamente que seu último bit é **0**. Nesse caso, a QBER é 0% e eles podem prosseguir, **descartando o bit anunciado**, com a chave preliminar restante sendo 110.

E se Eve tivesse tentado espionar a mensagem?

Suponha que, no nosso exemplo, Eve interceptou o segundo qubit (enviado por Alice como $|+\rangle$ na base diagonal).

Eve não sabia a base correta e mediu na base retilínea, obtendo $|1\rangle$ aleatoriamente. Ela então reenviou $|1\rangle$ para Bob que, por sua vez, também escolheu a base errada (retilínea) e mediu o estado $|1\rangle$, obtendo o bit 1.

Na etapa de filtragem, como Alice e Bob utilizaram bases diferentes para esse qubit, o resultado seria descartado. A presença de Eve não teria sido notada neste caso específico.

No entanto, considere o oitavo qubit. Alice envia $|+\rangle$ (bit 0 na base diagonal). Suponha que Eve meça na base retilínea e obtenha $|0\rangle$, reenviado para Bob.

Bob, que escolheu a base correta (diagonal), mede o estado $|0\rangle$ e obtém aleatoriamente $|+\rangle$ (bit 0) ou $|-\rangle$ (bit 1), com 50% de probabilidade cada.

Se ele obtiver $|-\rangle$ (bit 1), sua chave peneirada será 1101, enquanto a de Alice é 1100. Ao compararem o último bit, eles detectarão uma discordância, revelando a presença de Eve.

Qualquer tentativa de Eve de ganhar informação introduz um risco de ser detectada, e a QBER quantifica esse risco.

Com a segurança da chave verificada, Alice e Bob ainda precisam realizar duas etapas finais de processamento clássico para garantir que a chave seja perfeitamente secreta e utilizável.

4.2.5 Etapa 5: Reconciliação e Amplificação de Privacidade

Após a etapa de detecção de espionagem, Alice e Bob compartilham uma chave que eles sabem ser, com alta probabilidade, secreta. Porém, a chave ainda pode conter pequenas discrepâncias devido a imperfeições no canal quântico ou nos detectores, mesmo na ausência de Eve.

Além disso, a comparação pública de uma amostra da chave, embora necessária para estimar a QBER, vazou alguma informação para Eve. As duas sub-etapas finais, ambas realizadas classicamente, resolvem esses problemas para produzir uma chave final curta, perfeitamente aleatória e secreta.

A primeira é a chamada **Reconciliação de Informação (Correção de Erros)**: Alice e Bob aplicam um protocolo clássico de correção de erros em suas chaves restantes, por meio do canal público, para identificar e corrigir quaisquer bits discordantes.

Um método comum é Alice dividir sua chave em blocos e anunciar publicamente a paridade (soma dos bits módulo 2) de cada bloco. Bob compara a paridade de seus blocos correspondentes.

Se uma paridade divergir, eles sabem que há um erro naquele bloco e podem usar um processo de bissecção recursiva para localizar e corrigir o bit errado, ao custo de revelar um pouco mais de informação.

Este processo garante que ambos terminem com chaves idênticas, mas vazava alguma informação adicional para Eve sobre a chave.

Então, eles devem seguir para a etapa de **Amplificação de Privacidade**: para eliminar qualquer informação parcial que Eve possa ter adquirido, seja durante a

transmissão quântica ou durante a reconciliação de informação, Alice e Bob realizam a amplificação de privacidade.

Eles aplicam uma **função de hash universal** (escolhida publicamente de uma família de funções hash) às suas chaves idênticas. O resultado é uma chave final significativamente mais curta, mas da qual a informação mútua de Eve se torna pequena, tornando-a criptograficamente segura.

Ao final deste processo de cinco etapas, Alice e Bob compartilham uma chave secreta final. Com essa chave em mãos, eles podem agora usar um criptossistema simétrico comprovadamente seguro, como o one-time pad, para comunicar suas mensagens com sigilo absoluto.

Destacamos novamente que, com isso, a segurança da comunicação não depende mais da dificuldade de um problema matemático, mas das leis da física quântica. Isso faz com que nem mesmo um computador quântico consiga quebrar a criptografia, já que mesmo eles precisariam medir o sistema para interceptar informações.

4.3 Protocolo BB84 com Qiskit

Para ilustrar o funcionamento do protocolo BB84 na prática, vamos criar uma simulação em Qiskit.

Nosso cenário será idealizado: não haverá ruído no canal quântico e nem a presença da espiã Eve. O objetivo é demonstrar como Alice e Bob, partindo de sequências aleatórias, chegam a uma chave secreta compartilhada e idêntica.

A simulação envolverá quatro etapas principais:

- **Preparação de Alice:** Alice decide a sequência de bits e as bases que utilizará para codificar seus qubits.
- **Preparação de Bob:** Bob decide, de forma independente, a sequência de bases que usará para medir os qubits recebidos.
- **Codificação e Medição (Circuito Quântico):** um circuito quântico simulará o envio dos qubits de Alice e a medição por Bob, aplicando as portas lógicas correspondentes às bases escolhidas (Z para $+$ e X para $\times$).

- **Filtragem (Pós-processamento Clássico):** Alice e Bob comparam publicamente suas bases e mantêm apenas os bits onde as escolhas coincidiram.

Primeiro, definimos o tamanho da chave que Alice tentará enviar e geramos suas sequências aleatórias de bits e bases. Bob, por sua vez, gera sua própria sequência aleatória de bases para medição.

Esta definição inicial, junto aos *imports* necessários, mostrado a seguir:

```
import numpy as np
from qiskit import QuantumCircuit, QuantumRegister,
ClassicalRegister
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
import matplotlib.pyplot as plt

# --- 1. Configuração ---
np.random.seed(42)
num_bits = 16

# --- Geração de sequências aleatórias para Alice ---
alice_bits = np.random.randint(2, size=num_bits)
alice_bases = np.random.randint(2, size=num_bits) # 0
para base Z (+), 1 para base X (x)

# --- Geração de bases aleatórias para Bob ---
bob_bases = np.random.randint(2, size=num_bits)

print(f"Bits de Alice: {alice_bits}")
print(f"Bases de Alice: {alice_bases} (0=Z, 1=X)")
print(f"Bases de Bob: {bob_bases} (0=Z, 1=X)")
```

Código-fonte 6 - Protocolo BB84: Setup
Fonte: Elaborado pelo autor (2025)

Agora, criamos um circuito quântico para simular o processo. Para cada bit, o circuito fará o seguinte:

- Se a base de Alice for a base Z (representada por 0), codificamos o bit de Alice diretamente: $|0\rangle$ para bit 0 e $|1\rangle$ (aplicando uma porta X) para bit 1.
- Se a base de Alice for a base X (representada por 1), nós também aplicamos Hadamard para mudar da base computacional para X.

- Para a medição, se Bob escolher a base Z, ele mede diretamente. Se escolher a base X, ele aplica uma porta Hadamard antes da medição para voltar à base computacional.

```
def criar_circuito_bb84(alice_bits, alice_bases,
bob_bases):
    """Cria um circuito para cada bit do protocolo
    BB84."""
    circuitos = []
    num_bits = len(alice_bits)

    for i in range(num_bits):
        qc = QuantumCircuit(1, 1, name=f'qbit_{i}')

        # --- Preparação por Alice ---
        # Se o bit de Alice é 1, aplica porta X
        if alice_bits[i] == 1:
            qc.x(0)
        # Se a base de Alice é X, aplica Hadamard
        if alice_bases[i] == 1:
            qc.h(0)

        qc.barrier()

        # --- Medição por Bob ---
        # Se a base de Bob é X, aplica Hadamard
        if bob_bases[i] == 1:
            qc.h(0)

        # Bob mede na base computacional
        qc.measure(0, 0)
        circuitos.append(qc)

    return circuitos

# Cria uma lista de circuitos, um para cada qubit
circuitos_bb84 = criar_circuito_bb84(alice_bits,
alice_bases, bob_bases)
```

Código-fonte 7 - Protocolo BB84: Circuito

Fonte: Elaborado pelo autor (2025)

Executamos cada circuito no simulador do Qiskit Aer para obter os bits medidos por Bob:

```

simulator = AerSimulator()
bob_bits = []

for qc in circuitos_bb84:
    # Executa cada circuito com um único 'shot', pois é
    um processo determinístico
    job = simulator.run(qc, shots=1, memory=True)
    result = job.result()
    medida = int("".join(result.get_memory()))
    bob_bits.append(medida)

print(f"Bits de Bob:      {np.array(bob_bits)}")

```

Código-fonte 8 - Protocolo BB84: Simulação
Fonte: Elaborado pelo autor (2025)

Finalmente, realizamos a etapa clássica de filtragem no código-fonte a seguir. Alice e Bob comparam publicamente suas sequências de bases e mantêm apenas os bits onde as bases coincidiram.

```

# Alice e Bob comparam as bases
sifted_key_alice = []
sifted_key_bob = []

for i in range(num_bits):
    if alice_bases[i] == bob_bases[i]:
        sifted_key_alice.append(int(alice_bits[i]))
        sifted_key_bob.append(bob_bits[i])

print("\n--- Pós-processamento ---")
print(f"Bases de Alice:  {alice_bases}")
print(f"Bases de Bob:    {bob_bases}")
print(f"Índices onde as bases coincidem: {[i for i, (b1,b2) in enumerate(zip(alice_bases, bob_bases)) if b1==b2]}")

print(f"\nChave filtrada de Alice: {sifted_key_alice}")
print(f"Chave filtrada de Bob:    {sifted_key_bob}")

# Verificação final
if sifted_key_alice == sifted_key_bob:
    print("\nSUCESSO: As chaves peneiradas são idênticas.")
    print(f"Chave secreta final: {''.join(map(str, sifted_key_alice))}")
else:
    print("\nFALHA: As chaves peneiradas são diferentes. (Isso não deveria acontecer sem ruído/espião)")

```

Código-fonte 9 - Protocolo BB84: Sifting
Fonte: Elaborado pelo autor (2025)

Ao executar o código, o resultado esperado é que, para os índices onde as bases de Alice e Bob coincidem, seus bits sejam idênticos, resultando em uma chave secreta final compartilhada com sucesso. Um exemplo do output final está na figura:

```
— Pós-processamento —  
Bases de Alice: [1 0 1 1 1 1 1 0 1 0 0 1 0 0 0 1]  
Bases de Bob:   [1 0 1 1 1 1 0 0 1 1 0 0 1 0 1 1]  
Índices onde as bases coincidem: [0, 1, 2, 3, 4, 5, 7, 8, 10, 13, 15]  
  
Chave filtrada de Alice: [0, 1, 0, 0, 0, 1, 1, 1, 0, 0, 1]  
Chave filtrada de Bob:   [0, 1, 0, 0, 0, 1, 1, 1, 0, 0, 1]  
  
SUCESSO: As chaves peneiradas são idênticas.  
Chave secreta final: 01000111001
```

Figura 6 - Resultados do Protocolo BB84

Fonte: Elaborado pelo autor (2025)

Esta simulação demonstra a eficácia do BB84 para a distribuição segura de chaves em um cenário ideal.

5 O CENÁRIO PÓS-QUÂNTICO

Ao longo do capítulo, estudamos os dois lados da relação entre Computação Quântica e segurança da informação. Vimos como o Algoritmo de Shor representa uma ameaça fundamental à criptografia de chave pública, como o RSA, ao resolver eficientemente o problema da fatoração de inteiros e, em contrapartida, como os princípios da mecânica quântica oferecem uma nova promessa de segurança por meio da Distribuição de Chaves Quânticas, com o protocolo BB84.

A QKD, com sua segurança baseada nas leis da física, parece ser a solução definitiva. No entanto, a transição da teoria para a prática revela desafios e vulnerabilidades que não podem ser ignorados.

Implementações reais de protocolos como o BB84 precisam lidar com fontes de luz imperfeitas e outras limitações de hardware que podem ser exploradas por adversários. Diante desse cenário, uma iniciativa paralela e de extrema importância vem ganhando força: a **Criptografia Pós-Quântica** (PQC, Post-Quantum Cryptography).

A PQC não utiliza hardware quântico; em vez disso, busca desenvolver uma nova geração de algoritmos de criptografia clássicos que sejam resistentes tanto a ataques de computadores clássicos quanto quânticos. Essa abordagem tem a vantagem de poder ser implementada na infraestrutura digital que já utilizamos hoje.

Neste final de capítulo, olharemos para o futuro da segurança digital. Primeiramente, analisaremos as vulnerabilidades práticas da QKD e as contramedidas desenvolvidas para fortalecê-la.

Em seguida, introduziremos o campo da PQC, destacando a iniciativa de padronização do NIST e comparando as duas estratégias para proteger nossa informação na era quântica.

5.1 Vulnerabilidades da QKD

A segurança teórica de protocolos como o BB84 é forte, fundamentada em princípios quânticos como o Teorema da Não-Clonagem; ainda assim, a transição da

teoria para uma implementação física revela desafios e vulnerabilidades que surgem de imperfeições no hardware.

Sistemas reais não operam com as fontes de fótons e detectores ideais assumidos na teoria, e essas discrepâncias abrem brechas para ataques.

A principal dificuldade prática é a geração de fótons únicos sob demanda. Fontes ideais que emitem exatamente um fóton por pulso são tecnologicamente desafiadoras de construir.

Como alternativa, as implementações de QKD frequentemente utilizam pulsos de laser intensamente atenuados. Matematicamente, a distribuição de fótons nesses pulsos segue uma estatística de Poisson, o que significa que, embora a maioria dos pulsos contenha um único fóton (ou nenhum), uma pequena fração conterá dois ou mais fótons. Esta característica dá origem à vulnerabilidade mais conhecida da QKD prática: o **ataque de divisão de número de fótons**.

5.1.1 Photon Number Splitting

O **Ataque de Divisão de Número de Fótons** (PNS, Photon-Number-Splitting) explora diretamente a imperfeição das fontes de luz que emitem pulsos multi fotônicos. A estratégia de Eve é:

- **Monitoramento e Intercepção:** Eve monitora o canal quântico. Usando um equipamento que mede o número de fótons em um pulso sem perturbar o estado quântico (uma medição quântica não demolitiva, chamada QND), ela intercepta todos os pulsos que contêm mais de um fóton. Os pulsos de fóton único ela pode bloquear ou deixar passar, mas não pode medi-los sem arriscar ser detectada.
- **Clonagem Parcial:** para um pulso multi fotônico interceptado, Eve “rouba” um dos fótons e o armazena em uma memória quântica. Ela então reenvia o restante do pulso para Bob usando um canal ideal, sem perdas, para que ele não perceba a interceptação.
- **Coleta de Informação:** Eve aguarda pacientemente a comunicação pública entre Alice e Bob na fase de sifting. Quando eles anunciam as bases

corretas, Eve mede seus fótons armazenados na base correta e obtém informação sobre a chave secreta sem ter introduzido nenhum erro na sequência de Bob.

Este ataque quebra a premissa de segurança do BB84, que se baseia na detecção de erros (QBER) introduzidos pela espionagem. Como Eve só ataca pulsos que ela pode clonar parcialmente sem deixar rastros, a taxa de erro quântico permanece baixa, e Alice e Bob acreditam erroneamente que sua chave é segura.

5.1.2 Decoy States

Para mitigar o ataque PNS, a comunidade de criptografia quântica desenvolveu uma contramedida prática: o protocolo de **estados chamariz (decoy states)**. A ideia é permitir que Alice e Bob detectem a atividade de Eve mesmo sem saber o número exato de fótons em cada pulso, seguindo as etapas:

- **Variação de Intensidade:** Alice prepara seus pulsos de qubits (os estados de sinal) com uma certa intensidade média de fótons (μ). No entanto, ela intercala aleatoriamente esses pulsos com “estados chamariz”, que são pulsos preparados com intensidades diferentes e mais fracas (μ_1, μ_2 etc.).
- **Comunicação Pública Adicional:** após a fase de medição, durante a comunicação pública, além de revelarem as bases, Alice também anuncia qual pulso era de sinal e qual era chamariz (e com qual intensidade).
- **Análise Estatística:** Alice e Bob analisam separadamente as estatísticas (taxa de detecção e QBER) para os pulsos de sinal e para cada tipo de pulso chamariz. Como os pulsos chamariz são mais fracos, eles têm uma probabilidade muito menor de conter múltiplos fótons.
 - Se Eve estiver realizando um ataque PNS, ela irá preferencialmente bloquear ou alterar os pulsos chamariz mais fracos em comparação com os pulsos de sinal mais fortes. Essa diferença nas estatísticas entre os pulsos de sinal e os chamarizes revela sua presença.

Com os dados dos estados chamariz, Alice e Bob podem estimar com segurança o desempenho que teriam tido caso utilizassem uma fonte de fóton único ideal.

Isso lhes permite calcular um limite superior rigoroso para a quantidade de informação que Eve poderia ter obtido e, assim, aplicar a amplificação de privacidade de forma adequada para destilar uma chave segura.

A técnica de estados chamariz tornou as implementações de QKD com fontes de laser atenuadas significativamente mais seguras e práticas.

5.2 Criptografia Pós-Quântica

Enquanto a Distribuição de QKD oferece uma solução de segurança baseada em hardware quântico, uma iniciativa paralela de extrema importância vem ganhando força globalmente: a **Criptografia Pós-Quântica**.

A PQC representa uma abordagem de software, desenvolvendo uma nova geração de algoritmos criptográficos clássicos que são projetados para serem seguros contra ataques tanto de computadores clássicos quanto quânticos.

A principal motivação da PQC é criar sistemas de segurança que possam ser implementados na infraestrutura digital existente, sem a necessidade de hardware quântico especializado, como o exigido pela QKD.

A ideia é substituir algoritmos vulneráveis como o RSA por novos criptossistemas de chave pública que se baseiam em problemas matemáticos considerados difíceis para todos os tipos de computadores, incluindo os quânticos.

Esses problemas incluem, por exemplo, desafios baseados em reticulados (lattices), hashes e outros campos da matemática.

Dada a importância de uma transição segura e interoperável, o **Instituto Nacional de Padrões e Tecnologia** (NIST) dos Estados Unidos iniciou um processo global em 2016 para solicitar, avaliar e padronizar um ou mais algoritmos PQC resistentes a ataques quânticos. Esse esforço colaborativo envolveu a comunidade de segurança de todo o mundo para garantir que os futuros padrões sejam robustos e confiáveis.

O processo culminou na publicação de novos padrões em agosto de 2024, como o **FIPS 203**, o **FIPS 204** e o **FIPS 205**, que especificam algoritmos para encapsulamento de chaves e assinaturas digitais derivados de propostas como **CRYSTALS-Dilithium**, **CRYSTALS-KYBER** e **SPHINCS+**.

Embora o padrão final para produtos comerciais ainda não esteja definido, agências como a **CISA** (Cybersecurity and Infrastructure Security Agency) já recomendam que as organizações comecem a se preparar para a transição.

A CISA desenvolveu um “**roadmap**” que orienta as organizações a inventariar seus sistemas criptográficos, testar os novos padrões em ambientes de laboratório e planejar a migração, a fim de mitigar os riscos associados ao avanço da Computação Quântica.

5.3 Comparação entre PQC e QKD

A PQC e a QKD são duas estratégias distintas para o mesmo objetivo: proteger a informação na era quântica. O quadro a seguir mostra um comparativo entre as duas, resumindo suas abordagens, nível de segurança, dificuldade de implementação e vantagens.

Estratégia	Abordagem	Segurança	Implementação	Vantagem
PQC	Software. Trata-se de algoritmos clássicos que podem ser executados em computadores convencionais.	Baseada na dificuldade computacional de problemas matemáticos que se acredita serem resistentes a ataques quânticos.	Pode ser integrada à infraestrutura de comunicação e internet existente através de atualizações de software e protocolos.	Flexibilidade, escalabilidade e menor custo de implantação imediato.
QKD	Hardware. Requer canais quânticos (como fibras ópticas para enviar fótons) e hardware especializado.	Baseada nas leis da física. Oferece segurança incondicional (teórica), pois qualquer tentativa de espionagem é detectável.	Mais complexa e custosa, necessitando de uma infraestrutura dedicada.	Nível de segurança teoricamente superior, não dependente de suposições sobre a dificuldade de problemas matemáticos.

Quadro 1 - Comparação entre PQC e QKD
Fonte: Elaborado pelo autor (2025)

O cenário futuro mais provável não é a vitória de uma abordagem sobre a outra, mas sim uma estratégia de segurança híbrida. A QKD pode ser utilizada para tarefas de altíssima segurança, como a distribuição de chaves entre data centers ou entidades governamentais, enquanto a PQC será a solução escalável para proteger a vasta maioria das comunicações digitais, desde o comércio eletrônico até a proteção de sistemas de IA.

EMANIP

CONCLUSÃO

Ao longo deste capítulo, exploramos a relação de mão dupla entre a Computação Quântica e a segurança da informação. Partimos da constatação de que grande parte da segurança digital contemporânea, exemplificada pelo criptossistema RSA, se baseia na dificuldade computacional de problemas como a fatoração de inteiros. Demonstramos que essa base, embora útil e forte contra computadores clássicos, torna-se frágil diante do poder da Computação Quântica.

Apresentamos a principal ameaça quântica por meio do **Algoritmo de Shor**, que resolve o problema da fatoração em tempo polinomial. Para construir o entendimento de Shor, passamos pelo **Problema de Simon**, que estabelece o paradigma de extrair períodos ocultos com vantagem quântica exponencial, utilizando a **Transformada de Fourier** para converter informações de fase em resultados mensuráveis.

Vimos que Shor generalizou essa ideia, reduzindo a fatoração a um problema de descoberta de período (**order-finding**), resolvido eficientemente pela **Estimativa de Fase Quântica**, uma aplicação direta da QFT. Assim, entendemos como um computador quântico de larga escala poderia quebrar as bases da criptografia de chave pública moderna.

Em contrapartida, também estudamos o uso da própria mecânica quântica para criar novos paradigmas de segurança. Focamos na Distribuição de Chaves Quânticas, exemplificada pelo **protocolo BB84**, que permite às duas partes gerar uma chave secreta com segurança garantida pelas leis da física.

Sua proteção deriva de princípios fundamentais, como o **Teorema da Não-Clonagem** e o fato de que a medição perturba um sistema quântico, tornando qualquer tentativa de espionagem detectável. Embora teoricamente inviolável, discutimos as vulnerabilidades de implementações práticas da QKD, como o ataque de divisão de número de fótons (**PNS**), e as contramedidas desenvolvidas, como os estados chamariz (**decoy states**).

Finalmente, analisamos o cenário **pós-quântico**, no qual a comunidade de segurança da informação se prepara para um futuro com computadores quânticos.

Apresentamos a **Criptografia Pós-Quântica** como uma alternativa de software, baseada em algoritmos clássicos projetados para resistir a ataques quânticos.

Destacamos a iniciativa global liderada pelo **NIST** para padronizar esses novos algoritmos, garantindo uma transição segura para a infraestrutura digital existente. A conclusão é que o futuro da segurança provavelmente será híbrido: a **QKD** será usada para tarefas de altíssima criticidade, enquanto a **PQC** fornecerá a segurança escalável para o dia a dia digital.

Com a compreensão das ameaças e promessas que a Computação Quântica traz para a segurança, estamos preparados para detalhar sua aplicação em outra área revolucionária.

No próximo capítulo, estudaremos o **Aprendizado de Máquina Quântico (QML)**, onde algoritmos quânticos podem otimizar e acelerar modelos de aprendizado de máquina, conectando diretamente os conceitos de segurança, criptoanálise e otimização, além de novamente retomar as bases que estudamos desde o início do curso, como a superposição e o emaranhamento.

REFERÊNCIAS

BENNETT, C. H.; BRASSARD, G. **Quantum cryptography: Public key distribution and coin tossing**. In: Proceedings of IEEE International Conference on Computers, Systems and Signal Processing. 1984.

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. **Introduction to Algorithms**. 4th ed. Cambridge: The MIT Press, 2022.

GISIN, N.; RIBORDY, G.; TITTEL, W.; ZBINDEN, H. Quantum cryptography. **Reviews of Modern Physics**, 2024.

MURTA, G. **Lecture notes: Security proofs of Quantum Key Distribution**. 2023. Disponível em: <<https://glauciamg.github.io/teaching/>>. Acesso em: 11 fev. 2026.

NIELSEN, M. A.; CHUANG, I. L. **Quantum Computation and Quantum Information**. 10th anniversary ed. Cambridge: Cambridge University Press, 2010.

POST-QUANTUM CRYPTOGRAPHY. **National Institute of Standards and Technology**. Disponível em: <<https://csrc.nist.gov/projects/post-quantum-cryptography>>. Acesso em: 11 fev. 2026.

Qiskit SDK API Documentation. **IBM Quantum Platform**. 2025. Disponível em: <<https://quantum.cloud.ibm.com/docs/en/api/qiskit>>. Acesso em: 11 fev. 2026.

REDDY M, S. K.; MOHAN B, C. **Comprehensive Analysis of BB84, A Quantum Key Distribution Protocol**. 2023. Disponível em: <<https://arxiv.org/abs/2312.05609>>. Acesso em: 11 fev. 2026.

WATROUS, J. Understanding Quantum Information and Computation. **arXiv**, 2025. Disponível em: <<https://arxiv.org/abs/2507.11536>>. Acesso em: 11 fev. 2026.

WILDE, M. M. **Quantum Information Theory**. 2nd ed. Cambridge: Cambridge University Press, 2017.